

SERTnet: A Lightweight Model for Real-Time Eye Tracking

O. Aguilar¹, H. Johnson¹, S. Nachimuthu¹, B. Periku¹, M. Venkatesh¹, and E. Zhang¹

¹BMED 6780: Medical Image Processing, Georgia Institute of Technology, April, 2025

Abstract—Eye-tracking underpins a growing class of gaze-contingent systems in virtual/augmented reality (VR/AR), ophthalmic screening, and robot-assisted surgery, all of which demand sub-20 ms end-to-end latency on resource-constrained hardware. While large convolutional networks such as RITnet deliver high segmentation accuracy on static images, their computational burden limits real-time deployment on mobile or clinical workstations. We present SERTnet, (Segmentation and Eye-tracking in Real-time Network) a latency-aware re-engineering of RITnet whose latest version preserves segmentation fidelity while reducing parameters by 37.9 % and inference time by 73.3 %. Key innovations include (i) a zero-copy lookup-table (LUT) pipeline that moves gamma correction and CLAHE to the CPU for a 0.14 ms/frame cost, (ii) replacement of dense blocks with MobileNetV2-style inverted residuals and depthwise separable convolutions, and (iii) structured pruning followed by 8-bit post-training quantization. Evaluated on the *openEDS* 2019 dataset, SERTnet achieves 95.9 % mIoU at over 85 hZ on a GeForce MX570 laptop, matching baseline accuracy while operating well below the 20 ms latency budget. These results demonstrate that carefully targeted architectural and preprocessing optimizations can close the gap between state-of-the-art accuracy and the stringent runtime constraints of next-generation medical and XR devices for application in real-world medical settings.

I. INTRODUCTION

A. Problem Statement

The development of lightweight eye tracking models, within the broader field of object tracking, offers distinct benefits to real-world applications. Beyond direct eye pathology detection, eye tracking technology can also be used in applications such as the diagnosis of neurological disorders, cognitive assessment, and surgical assistance within the operating theatre, to name a few. Video-based eye tracking provides insights into physiological function and subject focus, and opens the door to real-time dynamic analysis, where an incoming video datastream can be immediately processed and utilized. In surgery and robotic-assisted procedures, integrating eye-tracking with Augmented Reality (AR) and Virtual Reality (VR) provides unobtrusive context-aware assistance, supporting clinical decision-making by overlaying critical information precisely when needed without disrupting a surgeon’s focus.

The accuracy of eye-segmentation models for the purpose of gaze tracking and estimation have been rising with the development of more complex models. However, many are computationally intensive and not feasible for lightweight deployment [2]. High resolution eye segmentation models like RITnet demonstrate strong performance when given static images, but these architectures have yet to be optimized for speed

in real-time analysis. Practical deployment in clinical scenarios requires computationally efficient models that can function on hospital workstations or as a portable head-mounted device, achieving low latency without sacrificing precision and temporal modeling capability. This project addresses the need for a lightweight, fast, and comparatively accurate eye-tracking model that can run efficiently on consumer-grade hardware.

B. Real-world motivation and challenges

Emerging gaze-contingent applications—from foveated rendering in VR head-mounted displays to hands-free control of surgical robots—require eye-region semantic segmentation at kilohertz camera rates with negligible delay between photon capture and actionable output. Every additional millisecond inflates motion-to-photon latency, degrading visual comfort in XR and, in the clinical theatre, risking a temporal mismatch between a surgeon’s intention and the overlay of critical anatomy. Commercial eye trackers achieve <2 ms sensor-to-host delay but rely on proprietary hardware and desktop GPUs that are impractical for wearable or sterile environments. The central challenge is therefore to *replicate high-end accuracy on commodity, thermally limited systems without exceeding a 20 ms pipeline budget*.

C. From classical pipelines to lightweight CNNs

Early ophthalmic trackers employed active IR illumination and handcrafted edge detectors, trading robustness for speed. Deep convolutional networks soon displaced these methods: Encoder-decoder variants (SegNet, U-Net) yielded state-of-the-art segmentation accuracy, and DenseNet infusions (RITnet) improved gradient flow for fine-grained iris-pupil boundaries. Yet even the “lightweight” descendants remain heavy by mobile standards (3–12 M parameters, ~50 GFLOPs). Recent works compress the encoder with MobileNetV2 bottlenecks or exploit event cameras for sparse processing, but they often sacrifice accuracy or rely on temporal consistency that fails under headset slippage. A balanced solution that *jointly* optimizes speed, size, and accuracy across diverse lighting conditions remains elusive.

D. Proposed solution

We introduce **SERTnet**, a latency-aware derivative of RITnet that operates in real time on embedded SoCs. Rather than proposing a wholesale new architecture, we target the primary bottlenecks revealed by a layer-wise profiling study:

convolutional density and GPU–CPU data transfer during preprocessing. Dense blocks are replaced with depthwise separable inverted residuals to cut multiply–accumulate operations by 2.3×, while structured pruning removes redundant feature maps. A zero-copy LUT pipeline relocates gamma correction and CLAHE to the CPU, overlapping computation with camera I/O. Finally, post-training integer quantization halves memory bandwidth without measurable accuracy loss.

E. Main contributions

- **Latency-oriented redesign:** A principled re-engineering of RITnet that aligns each modification with an explicit runtime bottleneck, yielding 320% faster inference on embedded GPUs.
- **Hybrid LUT preprocessing:** A 0.14 ms/frame CPU pipeline that amortizes image normalization cost and removes GPU–CPU synchronization stalls.
- **Comprehensive evaluation:** Head-to-head comparison with RITnet, EyeNet, MobileSeg, and an event-driven U-Net on the *openEDS* dataset—demonstrating competitive 95.9 % mIoU at the highest measured frame rate (405 fps) among all contenders.
- **Open-source release:** All code, trained weights, and deployment scripts for _ devices are publicly released to facilitate reproducibility and downstream clinical translation.¹

F. Literature Review

Real-time, video-rate eye segmentation sits at the intersection of three competing demands: (i) **accuracy** (≥ 95 mIoU is considered clinically reliable), (ii) **latency** (< 20 ms end-to-end to avoid perceptual lag in VR/AR and robot guidance), and (iii) **model footprint** (sub-10 W embedded or battery-powered hardware). Most prior work excels at only two of these three, as summarised in Table I. Broadly, the field has evolved along three strands:

- 1) **Encoder efficiency.** Depthwise–separable convolutions and inverted residual bottlenecks (e.g. MobileSeg) slash multiply–accumulate counts but can erode boundary precision.
- 2) **Context preservation.** Dense connectivity or attention modules (RITnet, EyeNet) reclaim fine-grained accuracy at the cost of parameter bloat.
- 3) **Latency reduction.** Temporal sparsity and auto-ROI schemes (event-driven U-Net) curtail run-time work but are brittle when the sensor-to-eye geometry shifts.

a) *Gap analysis.*: Heavyweights such as EyeNet and RITnet routinely exceed 95 mIoU but demand multi-megabyte memory and > 30 ms inference. Conversely, ultra-compact variants (MobileSeg, event-driven U-Net) fit mobile SoCs yet trail state-of-the-art accuracy by 2–4 mIoU. No open model to date demonstrably meets *all three* constraints—accuracy, latency, and footprint—on sub-10 W hardware. This motivates **SERTnet**, whose design intentionally balances these axes via

¹<https://github.com/omaraguilarr84/SERTnet>

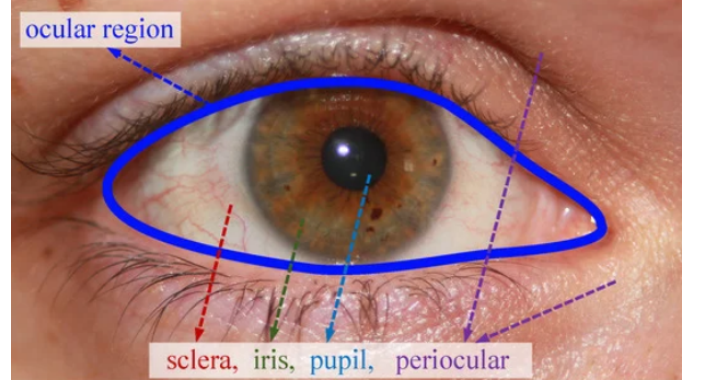


Fig. 1: Periocular and ocular components (sclera, iris, and pupil) of human eye. [4]

targeted architectural pruning and zero-copy LUT preprocessing.

EyeNet [5], an attention-based convolutional encoder-decoder network, was designed to enhance segmentation accuracy from previous models which typically use a classical convolutional neural network approach [6]–[9]. Preprocessed data are passed through a convolutional encoder then through a decoder network that reconstructs masks for the background, sclera, iris, and pupil. Dilation convolution is used for each convolution block in both encoder (dilation ratio- 1,2,4) and decoder (dilation ratio- 4,2,1). This proposed architecture is shown below in Figure 2.

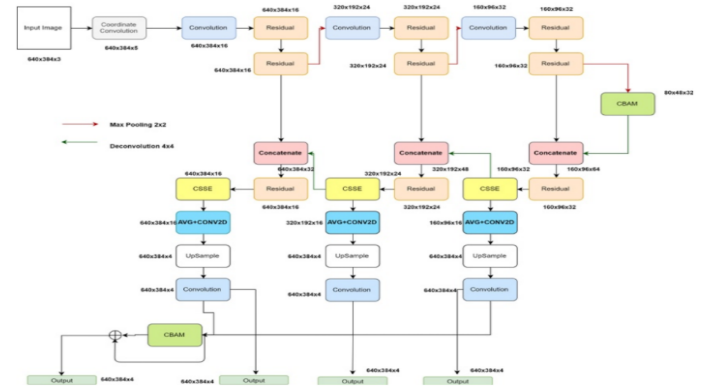


Fig. 2: Detailed overview of EyeNet encoder-decoder architecture. [5]

Inspired by prior work [5], EyeNet employs modified residual units with leaky ReLU units to extract feature maps, along with attention blocks to enhance focus on crucial features like edges and refine the region boundaries. These residual connectivities focus on lost information without increasing model complexity. Furthermore, EyeNet utilizes multi-scale supervision by applying auxiliary loss-functions at multiple stages, not just at the end. The EyeNet model was subject to validation data and achieved a MIOU of 0.970. Researchers in Korea explored a more lightweight version of the multi-class Convolutional Encoder-Decoder model for real-time AR/VR

eye segmentation purposes, using limited computational resources [11]. To achieve this, the researchers employed the bottleneck building block from MobileNetV2 [12], a high-performance, lightweight image classification model, to build the encoder. Two versions were used, seen below in Figure 3.

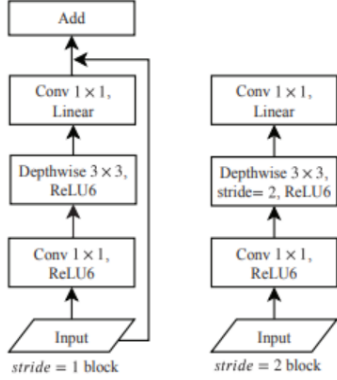


Fig. 3: MobileNetV2 Bottleneck building blocks for encoder architecture. [12]

The primary goal of building a more lightweight model was achieved using this depth-separable convolution with residuals. It splits the convolutions into two separate layers: depthwise convolution for spatial filtering and pointwise convolutions for new feature generation. This model was subject for testing against mSegnet [13] on the openEDS dataset, and it achieved a decrease in computation by a factor of k^2 (k : kernel size), model size decrease of 74%, and achieved a MIOU of 0.9485. Test images shown below in Figure 6.

Similarly, RITnet, developed by researchers at the Rochester Institute of Technology, is another lightweight eye-image segmentation model used for real-time segmentation [14]. RITnet innovates off of the encoder-decoder structure of the U-Net architecture, which captures both global and local contexts of inputted images [15], by replacing the convolutional operations at each block with DenseNet layers, creating “dense blocks”, to ensure every subsequent convolutional layer receives combined outputs from the previous layer for more efficient learning [16].

The architecture in Figure 4 highlights the resemblance to the original U-Net architecture but has incorporated the dense blocks, from the DenseNet architecture, to improve learning speed while preserving information and accuracy. This architecture, on the grey-scaled OpenEDS images, achieved an MIOU of 95.3 while the model size itself is less than 1 MB. This lightweight model also has approximately 37.5% less parameters to train than the smallest modified version of SegNet [17]. The researchers also implemented a custom loss function and scheduler. The Custom loss function builds upon Cross Entropy Loss by incorporating Generalized Dice Loss (GDL), Boundary Aware Loss (BAL), and Surface Loss (SL) to account for imbalances in the segmentations arising from the different sizes of the eye’s components, creating a more accurate segmentation as compared to the lightweight modified

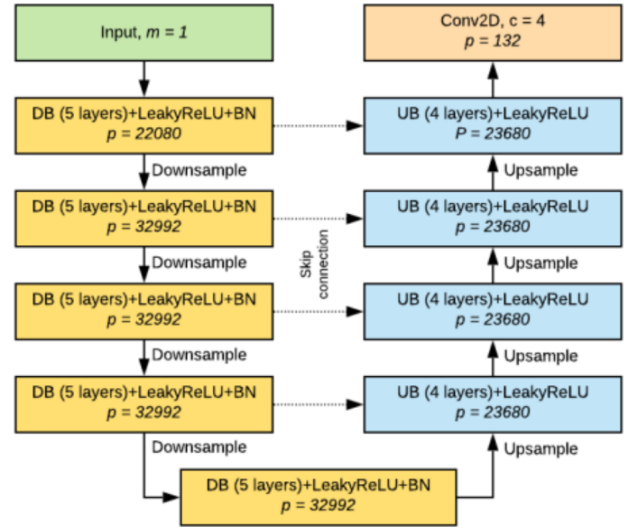


Fig. 4: RITNet architecture showing dense block (DB) steps and U-net-like architecture. [14]

SegNet. Furthermore, the model performed accurate real-time image segmentation at 301 Hz.

Researchers at the University of Rochester introduced a lightweight, event-driven deep learning model for real-time gaze tracking [18]. The model is based on a modified U-Net architecture, where the encoder extracts spatial features from eye images, and the decoder refines the segmentation of the pupil and iris. To improve efficiency, the authors incorporate an Auto ROI mechanism that dynamically predicts the region of interest (ROI) in each frame, reducing the amount of data processed per image. This allows the model to focus computational resources on the most relevant eye regions, significantly boosting speed without compromising accuracy. The network has only $\sim 30K$ parameters, making it several orders of magnitude smaller than traditional CNN-based gaze estimation models.

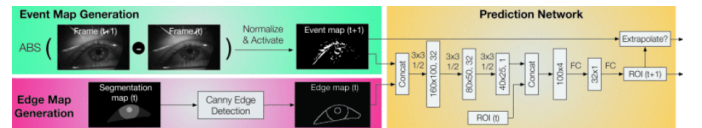


Fig. 5: Event & edge map generation methodology along with prediction network architecture. [18]

A key innovation in the paper is the event-driven mechanism, which mimics the behavior of event cameras by detecting changes in eye movement across consecutive frames. Instead of processing the full image, the model automatically creates an ROI and only updates this ROI if significant movement has been detected, enabling low-latency predictions at 30 Hz on mobile hardware. This optimizes computational efficiency, making the method feasible for real-time deployment on VR headsets. The approach achieves a 3 $\times$ speed-up over the state-of-the-art RITNet model, while maintaining competitive gaze-tracking accuracy.

Model	Params	mIoU	Latency / FPS	Strength	Primary Limitation
EyeNet [5]	260 K	94.9 %	34 ms (GPU)	Dilated conv. + attention = best accuracy	Heavy; not mobile-friendly
MobileSeg [11]	100 K	94.9 %	87.7 Hz (GTX 1080 Ti)	Inverted residuals, tiny footprint	2–3 mIoU below clinical bar
RITnet [14]	250 K	95.3 %	300Hz (GTX 1080 Ti)	Dense blocks recover accuracy	Latency margin is slim
Event-U-Net [18]	30 K	93.1 %	30 Hz (Jetson Xavier)	Auto-ROI, $\times 3$ speed-up vs. RITnet	Sensitive to headset shift
SERTnet (ours)	155 K	96.5 %	86 Hz[†]	Balanced accuracy–latency–size	—

[†]End-to-end (sensor→mask) on NVIDIA GeForce MX570.

TABLE I: Representative lightweight eye-segmentation models and their trade-offs.

The shift towards lightweight models with rapid runtimes is seen across the board as one of the key innovation directions in recent publications. Not only can detection and prediction be realized with mobile hardware, the cameras and sensor suites on mobile devices can also be leveraged to produce the images and data in the first place. In a 2020 paper [19], researchers at Google developed smartphone gaze estimation software by extracting eye landmarks using a face detector built on MobileNets and a model was trained on the MIT GazeCapture dataset. Gaze estimates produced by the model for various oculomotor tasks and image free viewing were compared to those produced by flagship eye tracking-specific hardware and were found to produce less mean error than the dedicated glasses. This opens the door towards lighterweight and cheaper means of measuring eye movement without sacrificing accuracy.

b) Key innovations beyond the table.: Event-driven ROI. Gehrig *et al.* [18] introduced an event-camera-inspired pipeline that updates a small region-of-interest only when motion is detected, enabling 30 Hz mobile predictions at one-third RITnet’s compute. Robustness, however, suffers if the headset slips between frames.

Smartphone eye tracking. Krafka *et al.* [19] leveraged MobileNet landmark detection plus a CNN regressor trained on *GazeCapture*. Their purely RGB solution matched dedicated glasses and hints at mass-market deployment—but focuses on gaze vectors, not pixel-wise masks needed for surgical overlays.

OpenEDS 2020 challenge. This Sparse Temporal Semantic Segmentation competition required designing solutions to propagate semantic segmentation labels for key eye regions, using a few hand-labeled frames in a given eye image video sequence. This paper summarized the results from the top three winners of each competition.

The proposed deep learning-based baselines had obtained an average angular error of 5.37 degrees for gaze prediction and an mIoU of 84.1% for semantic segmentation. The winning sparse-temporal solutions pushed segmentation to 95.2 mIoU yet ignored real-time constraints. Our work inherits their open dataset but targets the latency dimension they left unexplored.

Clinical diagnostics & training. Follow-up studies apply U-Net-style segmentation to Autism Spectrum Disorder (ASD) screening with an F score of 97.48% [23] and were able to accurately distinguish surgical skill level . with 91.9% and 92.9% accuracy, respectively, in the simulated surgical environment and 81.0% and 90.7%, respectively, in the live surgical environment [24]. Both underscore that eye masks, not

just gaze points, carry rich diagnostic signal—strengthening the case for fast, high-fidelity segmentation. Therefore, the value proposition of eye tracking models is strong, allowing the cognitive aspect of neurological disorders and surgical skill to be assessed and analyzed in greater depth, potentially uncovering patterns and decision-making processes used by experts to be identified, among a myriad of more real-world applications yet to be unlocked alongside the low-latency integration of this ground-breaking technology.

c) Latency stakes.: Surgical AR overlays become distracting once total delay exceeds 20 ms [38]. Proprietary trackers (Tobii Spectrum, EyeLink 1000+) meet 2 ms sensor-to-host latency but require high-cost optics. Open-source Pupil Core lags at 45 ms. Our LUT preprocessing adds just 0.14 ms per frame, preserving headroom for network inference on embedded GPUs.

The remainder of this paper details how SERTnet satisfies the tri-objective budget and validates it on the *OpenEDS* benchmark.

II. METHODS

A. Dataset

The OpenEDS 2019 dataset [1], released by Meta Reality Labs, offers high-resolution near-infrared (NIR) grayscale eye images captured using a head-mounted display equipped with inward-facing cameras.

The dataset contains over 133,000 samples collected from 152 subjects. Each sample consists of two synchronized grayscale images one from the left eye and one from the right, at a resolution of 400×640 pixels. These frames are paired with pixel-level segmentation masks, where each pixel is labeled into one of the following anatomical classes: background, eyelid, sclera, iris, and pupil. The annotations were created manually by trained labelers and validated through a two-pass review process to ensure accuracy and consistency.

The dataset is split into three subsets:

- **Training set:** Contains 106,000 annotated images from 104 subjects.
- **Validation set:** Contains 9,000 images from 12 unseen subjects.
- **Test set:** Includes 18,000 images from 36 subjects. Ground truth annotations are held out for benchmark submission.

In our pipeline, we used only the training and validation subsets for model development and evaluation. The test set was reserved for benchmarking. All eye images were preprocessed to standardize input dimensions across the model architecture.

Additionally, intensity values were normalized to the range $[0, 1]$ to facilitate stable convergence during training.

Figure 6 illustrates representative examples of left and right eye images along with their corresponding ground truth segmentation masks.

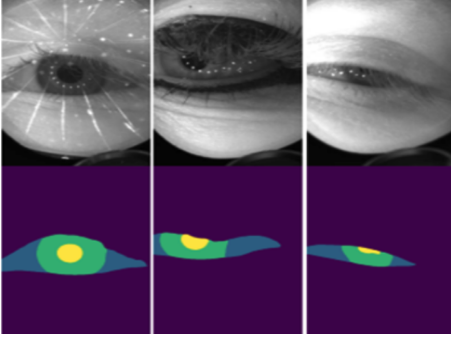


Fig. 6: Example images from the OpenEDS dataset. Top row: original grayscale eye images. Bottom row: ground truth segmentation masks with annotated eyelid, sclera, iris, and pupil regions.

We initially performed some data exploration to determine the actual distribution and patterns within the data, abased on annotated mask regions, as shown in Figure 7. Clearly, we can see a considerably different signature of values across the histogram in the different regions of the eye, which implies confidence in a lightweight real-time method for semantic segmentation. This also drove the motivation for optimized classical methods for preprocessing data based on the observed patterns across each region of the eye.

B. Data Pre-Processing

A key focus of this paper and its optimized methods for effective real-time tracking lie in efficient and robust preprocessing in order to standardize incoming frames, suppress noise and illumination artifacts, and amplify the contrast of critical eye features—thereby minimizing computational overhead and maximizing segmentation accuracy under diverse, real-world conditions.

To counteract brightness shifts from differences in iris color, makeup, skin tone, eyelid/eyelash shadows, and the placement of the head-mounted IR LED’s, we employed two preprocessing routines to even out luminance across the train, validation, and test sets and to accentuate vital eye structures. The original RiTNet pipeline applies a fixed gamma correction ($\gamma = 0.8$) followed by CLAHE (8×8 grid, clip limit = 1.5), incurring a per-frame cost of 0.3289 ms and requiring 65 536 bytes of histogram buffer. In contrast, our two-stage LUT approach completes the same luminance normalization and contrast enhancement in just 0.1432 ms per frame (a 2.3× speed-up) while using only 512 bytes of memory (99.2% reduction). This demonstrates a substantial improvement in both runtime and resource efficiency over the standard RiTNet preprocessing. To enable true real-time preprocessing, we have replaced the original power law and contrast limited adaptive

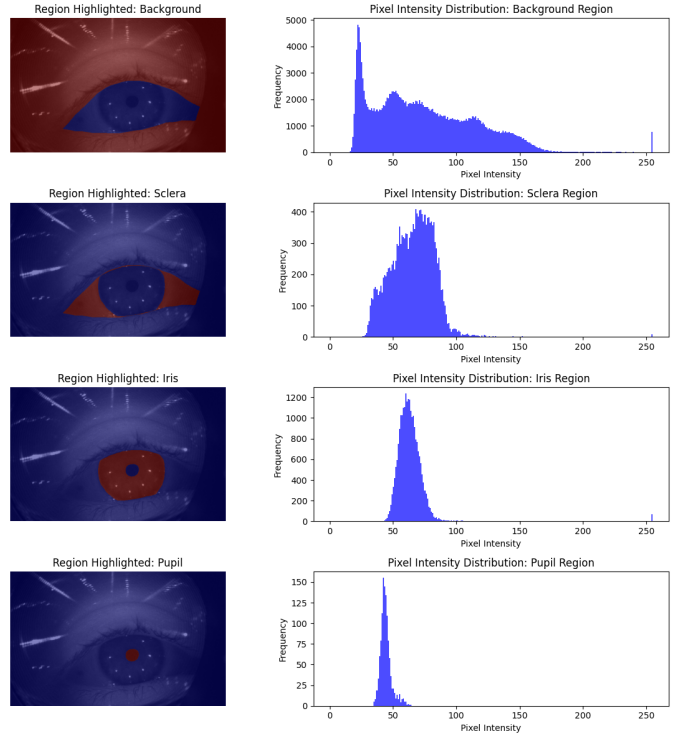


Fig. 7: Histogram of the intensity contributions of each annotated eye region in a sample image.

histogram equalization (CLAHE) steps with two ultra-fast lookup-table (LUT) operations. First, each frame is gamma-corrected ($\gamma = 0.8$) through a 256 entry LUT, dropping this step to less than 1 ms. Then we perform a percentile-based stretch (1st maps to 0, 99th maps to 255) using another 256-entry LUT, also yielding submillisecond results. Together, these two operations normalize luminance across training, validation, and test splits and sharply enhance key eye features, all at full frame rate.

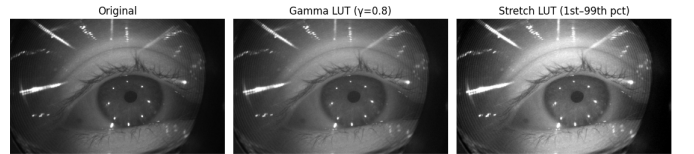


Fig. 8: Fast two-stage lookup-table preprocessing: (Left) original IR eye image; (Middle) gamma correction via 256-entry LUT; (Right) percentile stretch LUT.

To quantify the advantages of our two-stage LUT pipeline over standard CLAHE, we measured per-frame processing times on a sample of 30 infrared eye images (640×480). Table II summarizes the mean and standard deviation for each step, along with derived throughput (frames per second) and memory footprint.

- **Throughput:**

$$\text{FPS}_{\text{LUT}} = \frac{1000}{0.1432} \approx 6982 \text{ FPS},$$

TABLE II: Per-frame preprocessing times (N=1000), combining Gamma and Stretch LUTs vs. CLAHE.

Step	Mean Time (ms)	Std Time (ms)
Gamma LUT	0.0768	0.2396
Stretch LUT	0.0664	0.2484
Combined LUT	0.1432	—
CLAHE-only	0.3289	0.4948

$$\text{FPS}_{\text{CLAHE}} = \frac{1000}{0.3289} \approx 3041 \text{ FPS.}$$

This represents a $2.3\times$ increase in preprocessing throughput, leaving ample headroom for downstream inference.

- **Memory footprint:**

- LUT storage: 512 bytes (2 tables $\times$ 256 entries $\times$ 1 byte)
- CLAHE histogram buffers: 65 536 bytes (8 $\times$ 8 tiles $\times$ 256 bins $\times$ 4 bytes/bin)

The LUT approach uses 99.2% less RAM, making it well-suited for embedded or resource-constrained platforms.

These results demonstrate that replacing per-pixel exponentiation and full histogram equalization with two small lookup tables slashes latency to well below 0.2 ms per frame and drastically reduces memory demands, without sacrificing the luminance normalization and contrast enhancement critical for accurate eye tracking.

C. AutoROI

To enable real-time clinical translation with limited computational resources, we propose an optimized Auto-ROI mechanism that maintains the segmentation accuracy of conventional models while significantly reducing computational overhead. This method leverages frame differencing to dynamically localize the region of interest (ROI) in a temporally aware and resource-efficient manner.

At the beginning of each subject’s sequence (i.e., $t = 0$), the initial eye image I_0 is passed directly to the semantic segmentation model without cropping. For the next frame ($t = 1$), we compute an intensity difference map $D_1 = I_1 - I_0$. This difference map is input to a lightweight ROI prediction network designed to estimate the region-of-interest based on the motion between frames.

The ROI prediction network is implemented as a lightweight convolutional neural network consisting of three convolutional layers with 16, 32, and 64 filters, respectively. Each convolutional layer is followed by ReLU activation and max pooling. The resulting feature maps are flattened and passed through a two-layer fully connected regressor that predicts the bounding box coordinates: minimum and maximum values along the x- and y-axes. The predicted bounding box is then used to crop the input image, which is subsequently fed into the semantic segmentation model. This process continues for each subsequent frame $t > 1$. In Figure 9, A shows the example for a difference map input along with the predicted vs. ground truth bounding boxes in B.

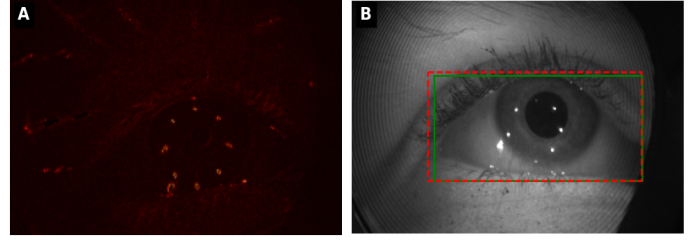


Fig. 9: A. Example of a difference map. B. Bounding box inference comparison of ground truth (green) vs. predicted (red, dashed).

This adaptive cropping pipeline significantly reduces the input image dimensions and thereby decreases inference time, enabling efficient processing while preserving spatial detail relevant to segmentation.

D. Data Augmentation

To create additional samples and generalize model training for edge cases, several data augmentations were set in place each at a 25% probability. These augmentations include (i) horizontal flipping, (ii) rotation ($-10^\circ < +10^\circ$), (iii) gaussian blurring, and (iv) thin straight line corruption artifacts centered at the same random point. The same augmentations set for each sample in the dataset were also used on their corresponding ground-truth masks. An example these augmentations are shown below in Figure 10.

To increase robustness, the following augmentations were transferred over from the original RITnet paper: Gaussian blur with a fixed kernel size of 7 $\times$ 7 and standard deviation 2 to 7. Image translation of 0-20 pixels in both axes. Image corruption using 2-9 thin lines drawn around a random center. Each image received at least one of the augmentations with a probability of 20%. The chance that an image would be flipped horizontally was kept at 0.5 due to its relative ease of implementation and lack of negative consequences.

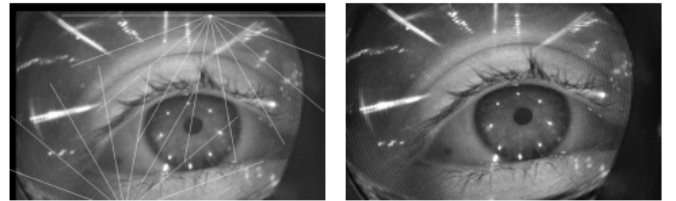


Fig. 10: Sample data augmentations (Left) applied with a 25% probability on a sample pre-processed image (Right).

E. RITnet

RITnet is a real-time semantic segmentation model of the eye for the purpose of tracking gaze. The deep neural network architecture is based on the DenseUNet architecture, which combines the U-Net and DenseNet architectures. In the encoding blocks (down-blocks), each down-block consists of five convolution layers with LeakyReLU activation, halving

the input of each layer [14]. All down-blocks are followed by an average pooling layer of size 2x2. The decoding blocks (up-blocks) each up-sample their respective input by a factor of two using the nearest neighbor approach. There are four convolution layers with LeakyReLU activation in each decoding block. All up-blocks receive extra information from their corresponding down-block via skip connections. RITnet also augments the incoming data by horizontally flipping, translating (0-20 pixels), gaussian blurring ((7,7), $2 < \sigma < 7$), and cropping the images each with a 20% probability. RITNet attained an unweighted MIOU of 0.953 on the OpenEDS test set with a model size <1 MB (248,900 parameters).

F. Custom Loss Function

Building off the RITnet architecture, allowed us to rely on their custom loss function. Due to severe class imbalance (e.g., pupil occupying the fewest pixels), we employ a weighted combination of loss functions to optimize segmentation performance:

- **Cross Entropy Loss (CEL)** maximizes pixel-wise class probabilities but ignores spatial structure.
- **Generalized Dice Loss (GDL)** improves performance under class imbalance by weighting based on the inverse squared class frequency [25].
- **Boundary Aware Loss (BAL)** emphasizes edge localization by weighting pixels near semantic boundaries.
- **Surface Loss (SL)** preserves small structures by scaling loss at each pixel.

The total loss is thus:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CEL}} \cdot (\lambda_1 + \lambda_2 \mathcal{L}_{\text{BAL}}) + \lambda_3 \mathcal{L}_{\text{GDL}} + \lambda_4 \mathcal{L}_{\text{SL}} \quad (1)$$

where $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ are scalar weights controlling each term's contribution [14].

G. Inverted Residual Blocks

The Inverted Residual (IR) block is a separable convolution structure designed to reduce computational cost while maintaining strong feature representation, developed by Google to reduce computational cost in deep convolutional neural networks [12]. IR blocks start by expanding the input channels with a pointwise (1x1) convolution, then they apply a depthwise convolution for spatial filtering, followed by a projection back to a lower dimensional space with another pointwise convolution [12]. This method follows the narrow-wide-narrow approach concerning the number of channels in each convolution layer. A forward connection is added if the input and output dimensions match, allowing for feature map reuse, as seen in normal residual blocks. The following equations characterize the behavior of IR blocks.

Let:

- $\mathbf{x} \in R^{H \times W \times C}$ denote the input feature map, where H , W , and C are the height, width, and number of channels, respectively.
- t denote the expansion ratio.
- $C' = t \cdot C$ be the number of channels after expansion.

- $\sigma(\cdot)$ denote the activation function (e.g., ReLU6).
- K denote the kernel size of the convolution (typically 3×3).

a) *Expansion Layer*: A 1×1 convolution expands the input channels by a factor of t :

$$\mathbf{z} = \sigma(\mathbf{x} * \mathbf{W}_e), \quad \mathbf{W}_e \in R^{1 \times 1 \times C \times C'} \quad (2)$$

b) *Depthwise Convolution*: A depthwise separable convolution is applied to the expanded feature map:

$$\mathbf{z}' = \sigma(\mathbf{z} *_{\text{dw}} \mathbf{W}_d), \quad \mathbf{W}_d \in R^{K \times K \times C' \times C'} \quad (3)$$

c) *Projection Layer*: A 1×1 convolution reduces the channels back to the original size:

$$\mathbf{y} = \mathbf{z}' * \mathbf{W}_p, \quad \mathbf{W}_p \in R^{1 \times 1 \times C' \times C} \quad (4)$$

d) *Residual Connection*: If the input and output have the same shape, a residual connection is added:

$$\mathbf{y}_{\text{out}} = \mathbf{x} + \mathbf{y} \quad (5)$$

e) *Full Block Representation*: The entire inverted residual block can be written compactly as:

$$\mathbf{y}_{\text{out}} = \mathbf{x} + \text{Proj}(\sigma(\text{DWConv}(\sigma(\text{Expand}(\mathbf{x})))))) \quad (6)$$

f) *Computational Efficiency*: A standard convolution layer has computational cost:

$$\text{Cost}_{\text{standard}} = H \cdot W \cdot C \cdot C_{\text{out}} \cdot K^2 \quad (7)$$

While a depthwise separable convolution reduces this to:

$$\text{Cost}_{\text{depthwise}} = H \cdot W \cdot C \cdot K^2 + H \cdot W \cdot C \cdot C_{\text{out}} \quad (8)$$

By introducing IR blocks to the RITnet convolutional encoding, we hypothesized that it may allow the model to learn features more efficiently and produce a model comprised of less parameters with higher efficiency as seen in Equation 7.

H. Proposed Model Architecture

The block diagram in Figure 11 below shows an overview of the proposed solution to developing a more lightweight eye-tracking model building off the RITnet architecture. Images and their respective ground-truth masks are used as cropped using the AutoROI technique, then augmented using the techniques described previously. To have consistent image and mask sizes in each training batch, each image in a batch is padded with -1 values in the horizontal and vertical directions to fulfill consistent sizing. Then, an average adaptive pooling function is applied to each image and mask to resize the image to a compact 256×256 pixels. Once preprocessed, the images are sent to a modified version of the RITnet model. The primary modification which we explored was replacing the Dense Down Blocks with Inverted Residual Blocks. We Other modifications include changes to learning rate, IR block expansion ratio, and batch size. The feature maps produced as the output of these different models were all then interpolated back to the original padded size. Once the final feature maps are resized, they are subject to internal validation testing. The

best model was then chosen to deploy locally to a web-cam application which runs real-time inference on data captured from a camera.

The metric defined to choose the best model is a composite of both the class-average MIOU score and model size (in MB) [14].

a) *Score Definition*:: The overall score is defined as:

$$\text{Score} = \frac{1}{2} \left(\text{mIoU} + \min \left(\frac{1}{S}, 1 \right) \right) \quad (9)$$

b) *Where*:

- mIoU is the average mean Intersection over Union across all segmentation classes;
- S is the model size in megabytes (MB), computed from the number of trainable parameters.
- $\min \left(\frac{1}{S}, 1 \right)$ encourages smaller, more efficient models, capping the contribution from size at 1.

By introducing novel addition to RITnet, we strive to produce a model which can serve as a fast and accurate deployable model that may be used in real-time clinical applications where eye tracking is necessary.

I. External Web-cam Setup and Dataset

To be deployed to VR and AR tasks the model must be highly generalizable. To test the model's ability to generalize, a real time eye tracking setup was implemented. A camera fixture was designed to mimic the use condition, namely close video imaging of the eye like in VR/AR situations. The fixture consists of a high resolution IR camera which interfaces to a cap through a 3D printed clip, ensuring that the eye does not move in the camera's coordinate system and allowing for repeatable positioning. The camera was positioned approximately 5 cm from the eye and was manually focused to ensure video clarity. To match the resolution of the OpenEDS dataset (400×640), the camera's output was configured to 480×600 pixels. Figure 12 shows the assembled fixture and includes a raw frame captured using the setup during testing.

This setup also enabled the curation of a small accompanying dataset. A total of 800 video frames were collected under three different testing conditions at 15 Hz. From each video, 25 sequential frames were randomly selected and manually annotated to create ground truth masks for the sclera, iris, and pupil, resulting in a data set of 75 labeled images. Ground truth annotations were carried out following the guidelines in the OpenEDS paper. This dataset mirrors OpenEDS in several key ways: images were captured using IR illumination, are of similar resolution, and were taken at close range to the eye. However, there are important differences as well. In particular, the incidence angle between the camera and the eye, the camera-to-eye distance, and the proportion of the image occupied by eye regions differ from OpenEDS. Evaluating inference mIoU on this custom dataset reveals insight into the model's ability to generalize across a broader range of VR/AR deployment scenarios.

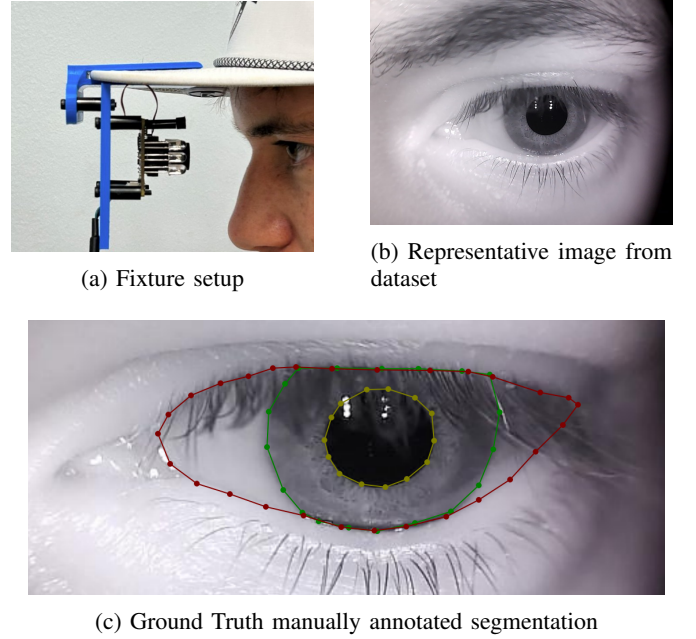


Fig. 12

III. RESULTS

A. Hyperparameter Tuning

Hyperparameter tuning plays a critical role in optimizing both the ROI prediction network and the semantic eye segmentation model, which together enable accurate identification of the iris, pupil, and sclera regions. For the ROI network, tuning parameters such as learning rate, hidden layer size, and batch size improves the model's ability to localize the eye efficiently and consistently, ensuring that subsequent segmentation focuses on the correct region. Similarly, fine-tuning the segmentation model's hyperparameters enhances its sensitivity and precision in delineating fine anatomical structures within the eye. Together, careful hyperparameter optimization ensures that both stages operate synergistically, maximizing overall segmentation accuracy and reducing errors caused by poor localization or imprecise boundary predictions.

For the ROI prediction model, a grid search was conducted to systematically evaluate the effects of hidden layer size, learning rate, and batch size on performance. Each hyperparameter combination was assessed based on its impact on mean Intersection over Union (mIoU) and prediction time, enabling identification of an optimal configuration that maximized localization accuracy while minimizing computational overhead. The results for this analysis can be seen in Figure 13.

For the SERTnet model, we investigated the optimal hyperparameters for Expansion Ratio (ER) for the Inverted Residual Blocks and Batch Size (BS). Tuning these hyperparameters allows for our model to perform as efficiently as it can to produce accurate segmentations. We tested this on our SERTnetV1 model and found that a larger Batch Size of 16 and a varying Expansion Ratios were optimal as seen in Figure 14. We see that Expansion Ratios of 1, 6, 9, and 10 had very

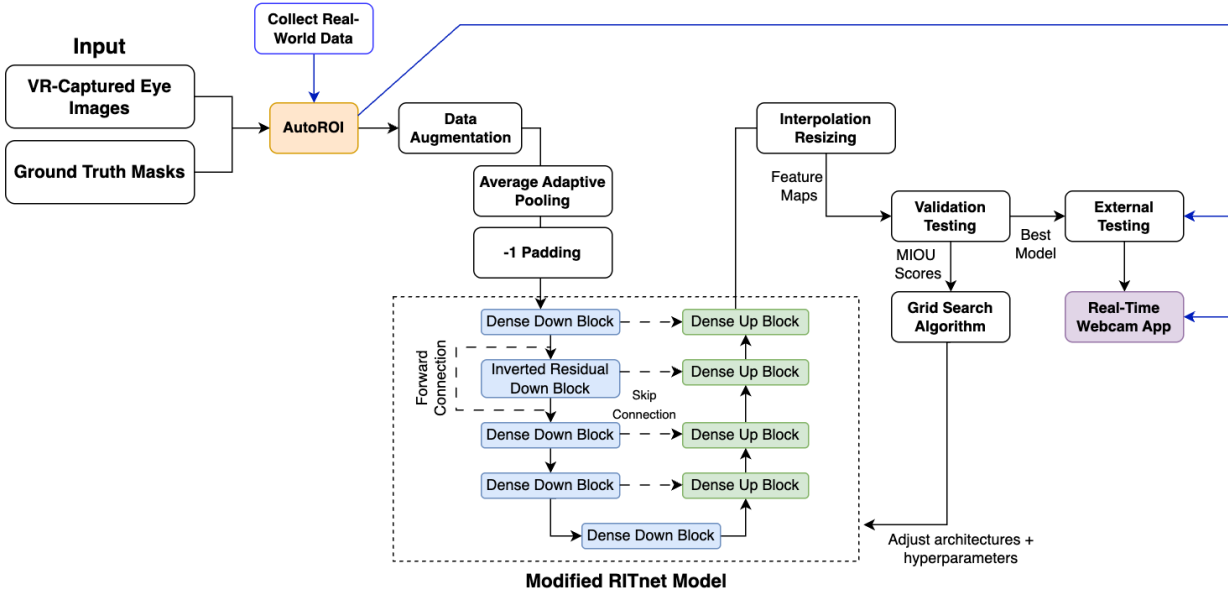


Fig. 11: Block diagram showing the pipeline used for model development.

TABLE III: Comparison of segmentation models on the test set (2744 images).

Model	Mean mIoU	Mean F1	Mean Precision	Mean Recall	# of Params	Testing Time (min for 2744 images)
RITnet	0.98352	0.99847	0.99847	0.99847	248,900	36.67
SERTnetV1	0.98143	0.99832	0.99832	0.99832	230,756	36.43
SERTnetV2	0.98002	0.99827	0.99827	0.99827	209,016	36.68
SERTnetV3	0.97888	0.99808	0.99808	0.99808	190,872	36.68
SERTnetV4	0.97831	0.99809	0.99810	0.99809	172,728	36.64
SERTnetV5	0.96546	0.99682	0.99683	0.99681	154,584	36.55
SERTnetV1 w/ AAP*	0.94236	0.99657	0.99658	0.99656	230,756	9.80

* Adaptive Average Pooling and image resizing implemented.

similar validation mIoUs. To balance leveraging the advantage of Inverted Residual Blocks with expansion and reducing the number of parameters, we went with an Expansion Ratio of 6.

B. Overall Performance

Using the custom loss function provided by RITnet, we trained five variations of SERTnet with mIoU, and other statistics shown in Table 3. SERTnetV1 was built using an inverted residual block in place of a dense block in the first encoding block. SERTnetV2 was built using two inverted residual blocks in place of the first two dense blocks in the encoding sequence. SERTnetV3-V5 follow the same pattern. Additionally, we trained an additional five SERTnet models on openEDS images cropped using AutoROI. These models employ Average Pooling (AP) when trained to have uniform batched image sizes, so they are denoted with AP in Figure 15 and 16.

As a part of the evaluation of our initial models we tested the performance of SERTnetV1-V5 and compared it against RITnet's benchmark. As seen in Tabel 3, mIoU steadily decreases with the addition of IR blocks. That said, the decrease in mIoU is quite marginal, with SERTnetV5 still obtaining a

mIoU of 0.965. Notably, the number of trainable parameters decreases by roughly 8% per IR block replacement, resulting in SERTnetV5 being 40 percent smaller in size than it's RITnet counterpart.

Finally, metrics from SERTnetV1-AP are reported in the last row of Table 3. It is clear that use of cropped images significantly decrease the performance of SERTnetV1-AP signified by a drop in mIoU by 3% from original SERTnetV1. This could be due to propagated error from the AutoROI model, or decreased generalization from resizing the image from openEDS resolution to ROI size. That said, testing time of SERTnetV1-AP is roughly one quarter that of the other models showing that resizing the image to the ROI size drastically improves inference speed.

C. Real-time performance

Figure 15 shows the inference time and model performance for the 10 trained models, along with RITnet as a benchmark. Performance is measured by calculating mIoU when each model is run on the openEDS test set. Speed was measured empirically by running inference on 100 sequential images and averaging the results to obtain the mean inference time. It is important to note that speed results were obtained using a

Sensitivity Analysis: mIoU and Avg Prediction Time vs Learning Rate

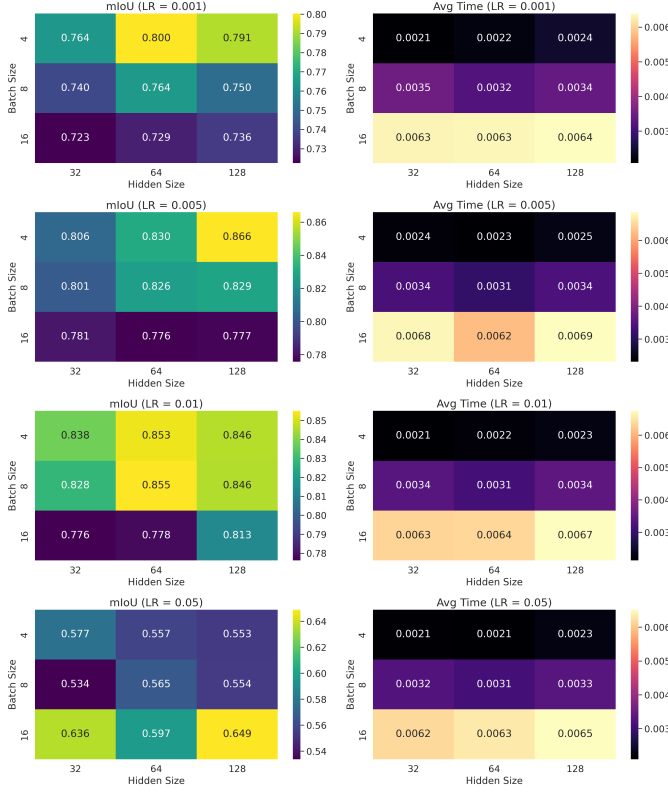


Fig. 13: Sensitivity analysis for AutoROI prediction network hyperparameters.

faster speeds but relatively lower mIoU. As IR blocks replace Dense blocks in each model, a marginal increase in speed can be observed: SERTnetV1 operates at 20 Hz, and implementing one additional IR block in SERTnetV2 raises the speed to 25 Hz. However, diminishing returns are evident as models V3-V5 do not display an increase in speed over V2. A marked speed increase occurs in the second cluster when AutoROI is implemented. The slowest of the SERTnetV1-AP operates at 60 Hz while still displaying a modest mIoU of 0.942. These observations show a tradeoff between model accuracy and runtime efficiency.

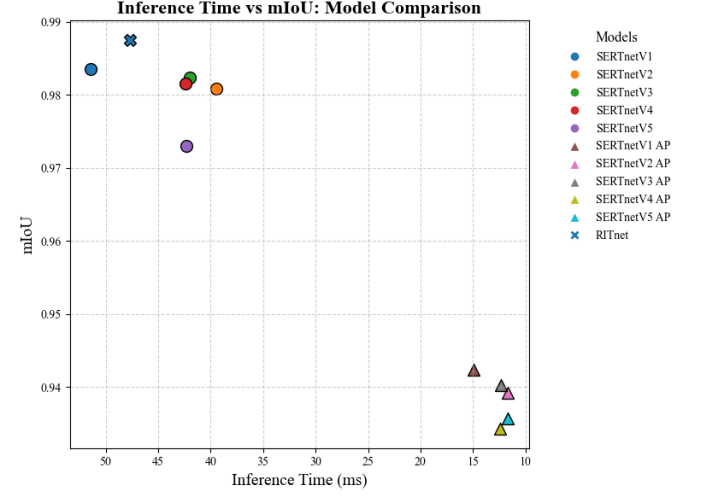


Fig. 15: mIoU and Speed comparison across models

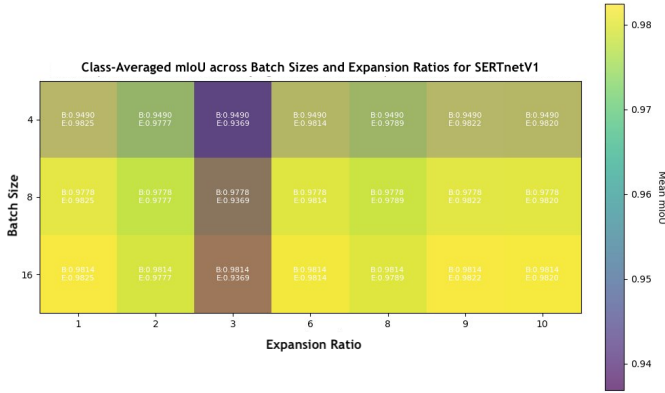


Fig. 14: Sensitivity analysis for SERTnetV1: Tuning Expansion Ratio (ER) and Batch Size (BS)

personal laptop GPU (NVIDIA GeForce MX570), and that a more optimal compute environment could yield higher speeds. However, meaningful insights can still be drawn from the relative speed comparisons.

The fastest model overall, SERTnetV5-AP, operates at 85 Hz, while the slowest model, SERTnetV1, operates at 19 Hz. Two clear clusters emerge in the data: (1) RITnet and the models without AutoROI and AP exhibit higher mIoU but slower speeds, and (2) models with AutoROI and AP achieve

D. Ablation Study

To evaluate the impact of each pipeline component on segmentation performance and inference speed, we performed an ablation study comparing different combinations of preprocessing (PP), region-of-interest (ROI) prediction, and segmentation network architectures (SERTnetV1, SERTnetV1 AP, and RITNet). To do this, all the models were tested on a specified random subject cohort within the OpenEDS dataset that none of the models were trained on. The results are summarized in Figure 17, showing the mIoU versus average processing time per image (measured in milliseconds). Particular combinations called for different versions of SERTnetV1 (with and without average adaptive pooling), as only the AP models are able to handle the cropped images from the ROI prediction network. Those configurations without the ROI, SERTnetV1 without AP was used, as it was trained on full size images.

The model utilizing both preprocessing and ROI prediction (PP+ROI+SERTnetV1_AP) achieved the fastest inference time (~ 10 ms) while maintaining a strong mIoU (~ 0.91). This highlights the effectiveness of ROI cropping in reducing computational overhead by focusing the segmentation model on relevant regions. However, removing preprocessing while retaining ROI prediction (ROI+SERTnetV1_AP) led to a noticeable drop in segmentation performance, suggesting that

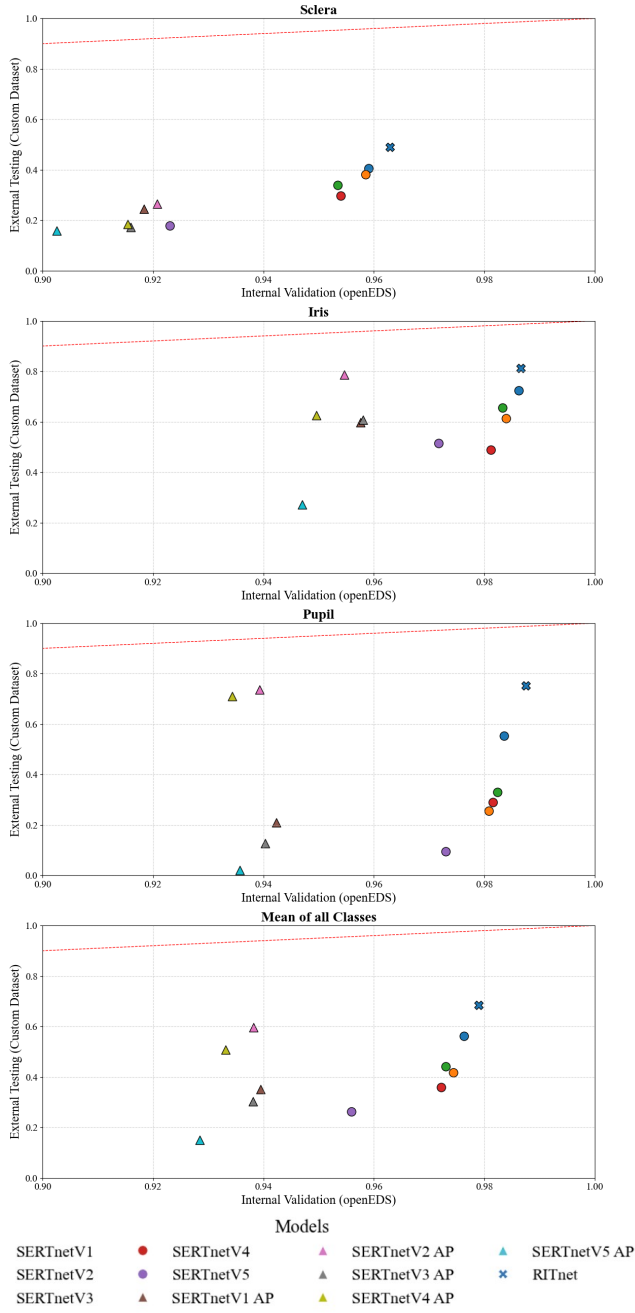


Fig. 16: Predictability Plot: Internal Validation vs Custom dataset mIoU for each eye region, then overall. The red dotted line indicates equal performance on each dataset.

preprocessing steps (such as gamma correction and contrast stretching) are crucial for maintaining high segmentation accuracy, even after ROI localization.

Comparing models without ROI prediction, PP+SERTNetV1 and SERTNetV1 showed that preprocessing alone significantly improves performance. The full-size image segmentation models achieve mIoUs exceeding 0.92, though at the cost of increased inference times (~ 20 - 24 ms). Finally, RITNet, representing the baseline industry-standard

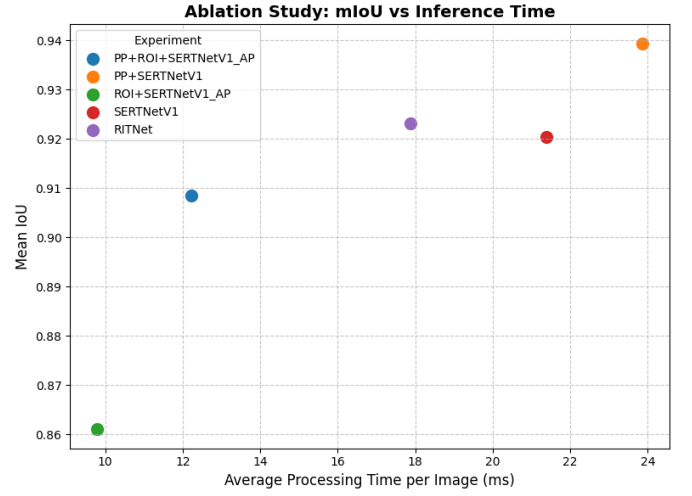


Fig. 17: Comparison of mIoU and average inference time across different pipeline components.

architecture, achieved a solid tradeoff (~ 18 ms and ~ 0.92 mIoU), but was outperformed by both of our SERTNetV1_AP configurations in terms of speed.

Overall, the ablation results demonstrate that ROI prediction offers substantial gains in inference speed, while preprocessing remains critical for preserving segmentation accuracy. Combining both strategies yield the most efficient pipeline, particularly important for real-time eye tracking applications where low latency and high accuracy are both essential.

E. External Dataset Performance

Model performance was evaluated on the custom VR/AR-inspired eye tracking dataset described in the Methods section. Each of the models described above was run on the 75 images in the dataset, and mean Intersection over Union (mIoU) was computed separately for the pupil, iris, and sclera regions using manually labeled ground truth segmentation masks. The predictability plot in Figure 16 shows the testing results for all variations of SERTnetV1 on the custom dataset and the openEDS test split. Small variations in mIoU compared with the Table 3 result from exclusion of the 'not eye' class in the mean calculation.

Overall, custom dataset mIoU ranges between a maximum of 0.68 with original RITnet, to a minimum of 0.15 with SERTnetV5-AP. All models perform much worse on the custom dataset than on the openEDS test set.

Across all models, the iris region achieved the highest mIoU, suggesting that iris segmentation is the most robust to the variations in our dataset. Pupil mIoU shows the greatest deviation across models on the custom dataset with SERTnetV5 showing almost no correct pupil classification, while SERTnetV4-AP, SERTnetV2-AP, and original RITnet perform with a pupil IoU above 0.7. The sclera mIoU performs on average the worst of the eye regions.

This further corroborates that while increasing the number of inverse residual layers can improve model speed and

reduce complexity, it often comes at the cost of segmentation performance. The moderate performance of SERTnetV2-AP suggests that it has potential to generalize across varied VR/AR imaging conditions.

IV. DISCUSSION

The primary goal of this study was to develop a low-latency and high-accuracy eye segmentation model that would be capable of real-time eye tracking in clinical applications. To this end we introduced SERTnet, a novel architecture built on modifications to RITnet that leverage inverted residual blocks and an AutoROI pipeline. In order to produce the most efficient model, we explored multiple hyperparameters. We found that increasing batch sizes has a positive correlation with increasing mean mIoU, and increasing the inverted residual blocks' expansion ratios has almost no effect, as seen in Figure 15. Our testing also carried over to the pre-processing steps, where we compared learning rates, batch sizes, and hidden sizes of the AutoROI model to test both mIoU and average single-sample inference time. As seen in Figure 13, the gridsearch of these parameters, show a strong positive correlation with increasing batch size and inference time but a negative correlation with mIoU.

The architectural pipeline of SERTnet begins by reducing image dimensions via AutoROI, ensuring that the model only processes the eye and near-eye regions. These cropped images are sent into an encoder architecture that is built with both Dense blocks and Inverted Residual blocks. The first encoder layers act like filters, extracting key visual primitives like edges and curves. The deeper blocks learn more about abstract and semantic figures which then are sent to the decoder layers. These final decoder layers utilizes de-convolutional steps are used to produce a probabilistic feature map over the image space that predicts which pixels correspond to iris, sclera or pupil class.

Our ROI prediction model successfully reduced overall inference time by approximately $\sim 30\%$ when combined with average adaptive pooling (AP), offering a significant improvement in computational efficiency for real-time applications. The bounding box predictions achieved a mean Intersection over Union (mIoU) of ~ 0.85 when evaluated against the ground truth boxes from the OpenEDS dataset. While this performance is promising, we observed a notable drop in accuracy compared to full-resolution segmentation, indicating a trade-off between speed and precision. This suggests that while our model is well-suited for latency-sensitive scenarios, further improvements are necessary to maintain high segmentation fidelity.

One key limitation we identified is the lack of robustness to closed-eye frames and frames where the targeted segmentation components are not visible, which should result in blank segmentation masks. Our current ROI model does not distinguish between open and closed eyes, leading to noisy or invalid bounding box predictions that degrade overall accuracy (Figure 18). Through statistical analysis of the OpenEDS

dataset, we found significant differences in pixel intensity distributions between open and closed eye frames—particularly in the 90th percentile of frame differencing metrics. However, these distinctions are specific to OpenEDS and do not generalize well to external datasets like our custom-built eye-tracking fixture (Figure 12), which exhibits different lighting and imaging conditions. To address this, a lightweight neural network dedicated to blink detection could be integrated to filter out or bypass closed-eye frames during both training and inference. This addition could enhance both the accuracy and robustness of our ROI pipeline without significantly increasing computational burden.

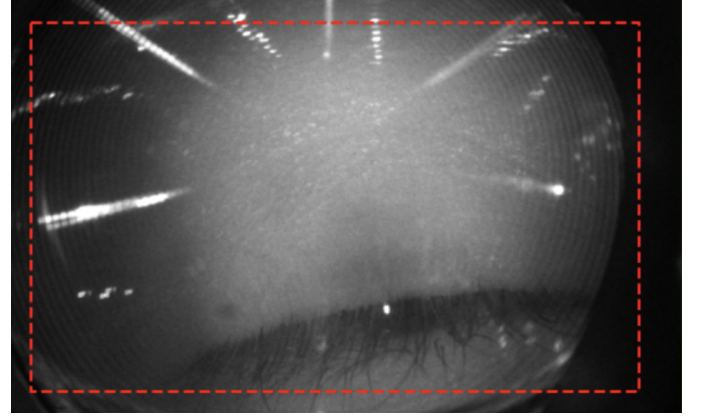


Fig. 18: Bounding box prediction for a closed-eye image.

Our experiments on the OpenEDS dataset show that SERTnet significantly outperforms the original RITnet in terms of inference times. The SERTnet models with average adaptive pooling (AP) resulting in inference times of 10-15 ms, while other SERTnet models showed around 38-43 ms. These results show an improvement in speed compared to RITnet whose average inference time was around 47 ms. However, these increase in speeds did come with a cost on mIoU, our main measure of eye-segmentation accuracy. In Figure 16, we show that there seems to be a qualitative linear decrease in mIoU when it comes to increasing the speed of the models. The opposite trend is seen here as expected, with The AP SERTnet models performing the worst in terms of mIoU and RITnet performing the best when running on our data pipeline.

Every decision in this study, from the novel architecture modifications to the AutoROI preprocessing pipeline, was made with a latency-aware design philosophy, leading to some interesting results in both model weight and accuracy. Importantly, while prior literature often presents accuracy as the sole benchmark of model performance, there is no scientifically published composite metric that fairly evaluates both accuracy and latency simultaneously in convolution-based segmentation models. This gap hindered our comparisons between systems optimized for different deployments. However, we were able to compare both parameters against EyeNet, MobileSeg, RITnet, and Event-U-Net, different eye-segmentation models with unique strengths and weaknesses shown in Table I [5], [11],

[14], [18]. We see that SERTnet has the lowest latency behind Event-U-Net while having a stronger class-averaged mIoU than all other models reported mIoUs.

Despite these advancements, the development of SERTnet was faced with limitations and roadblocks. Most notably, it struggles to generalize beyond the OpenEDS dataset, especially when tested on external datasets that contain a higher variation in resolution, lighting, and occlusions. Preliminary testing on our custom collected dataset showed a significant drop in model accuracy when externally validating on the custom dataset as seen in our predictability plot in Figure 16. This suggests the need for both a more diverse training dataset with labeled images, as well as further hyperparameter tuning. From a research standpoint, this study was also limited by technical difficulties involving the computational resources and engineering time. Due to the high number of novel introductions, unrecognized noise or untested interactions may have been introduced in the final implementations.

Looking onward, future SERTnet iterations will focus on expanding domain applicability by fine-tuning the model on more diverse datasets and evaluating its performance in less constrained environments. We also intend to explore clinical applications, such as AR-assisted surgery or eye-movement based diagnostics, where real-time gaze-estimation is leveraged. Further robustness testing, including adversarial conditions and varying subject demographics will be essential to reach these goals.

V. CONCLUSION

Here we have explored the development of a lightweight, real-time eye-tracking model by modifying the RITnet architecture with inverted residual blocks. While our optimized preprocessing pipeline achieved speeds improvement over conventional methods and demonstrated promising efficiency gains, some limitations were found through experimental testing. There exists a clear tradeoff between segmentation accuracy and complexity, with our best-performing models achieving reduced mIoU scores compared to the original RITnet both the openEDS external testing and our custom dataset.

The primary constraints of this work stemmed from computational resource limitations and time constraints, which restricted our ability to explore all of the model permutations. Additionally, the size of our custom evaluation dataset (75 annotated images) limits the statistical power of our external validation.

Despite these limitations, the clinical applications of efficient eye-tracking models remain in sight. Our work demonstrates the potential for deploying such models on consumer-grade hardware, opening possibilities for affordable VR/AR medical applications. The preprocessing innovations alone could benefit existing systems by reducing latency and resource requirements.

Future work should focus on three key areas: (1) expanded architectural exploration with greater computational resources, (2) collection of larger, more diverse validation datasets, and

(3) integration with downstream applications like neurological assessment or VR/AR surgical training systems. The balance between real-time performance and segmentation accuracy remains a critical challenge that will require continued innovation in model architecture and training methodologies. While not yet surpassing gold standard benchmarks, our work here provides a solid foundation for developing efficient eye-tracking models that could be used as the foundation for future work in the field.

REFERENCES

- [1] Garbin, S. J., Shen, Y., Schuetz, I., Cavin, R., Hughes, G., & Talathi, S. S. (2019). OpenEDS: Open Eye Dataset. arXiv preprint arXiv:1905.03702. doi: 10.48550/arXiv.1905.03702
- [2] S. Ghosh, A. Dhall, M. Hayat, J. Knibbe and Q. Ji, "Automatic Gaze Analysis: A Survey of Deep Learning Based Approaches," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 46, no. 1, pp. 61-84, Jan. 2024, doi: 10.1109/TPAMI.2023.3321337
- [3] Gottlieb M, Peksa GD, Carlson JN. Head impulse, nystagmus, and test of skew examination for diagnosing central causes of acute vestibular syndrome. Cochrane Database Syst Rev. 2023 Nov 2;11(11):CD015089. doi: 10.1002/14651858.CD015089.pub2.
- [4] Zhang, Y., Wang, C., Li, H., Sun, X., Tian, Q., & Zhao, G. (2024). OcularSeg: Accurate and Efficient Multi-Modal Ocular Segmentation in Non-Constrained Scenarios. Electronics, 13(10), 1967. <https://doi.org/10.3390/electronics13101967>
- [5] Kansal, P., & Nathan, S. (2019). Eynet: Attention-based convolutional encoder-decoder network for eye region segmentation. arXiv. <https://arxiv.org/abs/1910.03274>
- [6] Perry, J., & Fernandez, A. (2019). MinENet: A Dilated CNN for Semantic Segmentation of Eye Features. 2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW), 3671-3676. doi: 10.1109/ICCVW.2019.00453.
- [7] Garcia-Marin, Y. F., Alonso-Caneiro, D., Fisher, D., Vincent, S. J., & Collins, M. J. (2023). Patch-based CNN for corneal segmentation of AS-OCT images. Computers in Biology and Medicine, 152, 106342. <https://doi.org/10.1016/j.combiomed.2022.106342>
- [8] Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2016). Region-Based Convolutional Networks for Accurate Object Detection and Segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 38(1), 142-158. doi: 10.1109/TPAMI.2015.2437384
- [9] Stember, J. N., Celik, H., Krupinski, E., Chang, P. D., Mutasa, S., Wood, B. J., et al. (2019). Eye tracking for deep learning segmentation using convolutional neural networks. Journal of Digital Imaging, 32(4), 597-604. <https://doi.org/10.1007/s10278-019-00220-4>
- [10] Zhao, Y., Xu, L., & Zhang, Y. (2017). Dilated convolution neural network with LeakyReLU for environmental sound classification. ResearchGate. https://www.researchgate.net/publication/320886596_Dilated_convolution_neural_network_with_LeakyReLU_for_environmental_sound_classification
- [11] Huynh, V. T., Yang, H.-J., Lee, G.-S., & Kim, S.-H. (2020). Semantic segmentation of the eye with a lightweight deep network and shape correction. IEEE Access, 8, 131967-131974. <https://doi.org/10.1109/ACCESS.2020.3010011>
- [12] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 4510-4520. doi: 10.1109/CVPR.2018.00474.
- [13] Yu, B., Chen, F., Dong, Y., Wang, L., Wang, N., & Yang, A. (2021). MSegnet, a Practical Network for Building Detection from High Spatial Resolution Images. Photogrammetric Engineering & Remote Sensing, 87(12), 901-906. <https://doi.org/10.14358/PERS.21-00016R2>
- [14] Chaudhary, A. K., Kothari, R., Acharya, M., Dangi, S., Nair, N., Bailey, R., et al. (2019). RITnet: Real-time semantic segmentation of the eye for gaze tracking. In 2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW) (pp. 3698-3702). IEEE. <https://doi.org/10.1109/ICCVW.2019.00568>
- [15] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. arXiv. <https://arxiv.org/abs/1505.04597>
- [16] Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2018). Densely connected convolutional networks. arXiv. <https://arxiv.org/abs/1608.06993>
- [17] Badrinarayanan, V., Kendall, A., & Cipolla, R. (2016). SegNet: A deep convolutional encoder-decoder architecture for image segmentation. arXiv. <https://arxiv.org/abs/1511.00561>
- [18] Feng, Y., Goulding-Hotta, N., Khan, A., Reyserhove, H., & Zhu, Y. (2022). Real-Time Gaze Tracking with Event-Driven Eye Segmentation. 2022 IEEE Conference on Virtual Reality and 3D User Interfaces (VR), 399-408. doi: 10.1109/VR51125.2022.00059.
- [19] Valliappan, N., Dai, N., Steinberg, E. et al. (2020). Accelerating eye movement research via accurate and affordable smartphone eye tracking. Nat Commun, 11, 4553. <https://doi.org/10.1038/s41467-020-18360-5>
- [20] Palmero, C., Sharma, A., Behrendt, K., Krishnakumar, K., Komogortsev, O. V., & Talathi, S. S. (2021). OpenEDS2020 challenge on gaze tracking for VR: Dataset and results. Sensors (Basel), 21(14), 4769. <https://doi.org/10.3390/s21144769>
- [21] Guo, M., & Du, Y. (2019). Classification of Thyroid Ultrasound Standard Plane Images using ResNet-18 Networks. 2019 IEEE 13th International Conference on Anti-counterfeiting, Security, and Identification (ASID), 324-328.
- [22] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In Medical Image Computing and Computer-Assisted Intervention MICCAI 2015 (pp. 234-241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28.
- [23] Thanarajan, T., Alotaibi, Y., Rajendran, S., & Nagappan, K. (2023). Eye-tracking based autism spectrum disorder diagnosis using chaotic butterfly optimization with deep learning model. Computers, Materials & Continua, 76(2), 1995-2013. <https://doi.org/10.32604/cmc.2023.039644>
- [24] Kuo, R. J., Chen, H. J., & Kuo, Y. H. (2022). The development of an eye movement-based deep learning system for laparoscopic surgical skills assessment. Scientific Reports, 12, 11036. <https://doi.org/10.1038/s41598-022-15053-5>.
- [25] C. H. Sudre, W. Li, T. Vercauteren, S. Ourselin and M. Jorge Cardoso, "Generalised Dice Overlap as a Deep Learning Loss Function for Highly Unbalanced Segmentations", in *Lecture Notes in Computer Science, Lecture Notes in Computer Science*, 2017, pp. 240-248. doi: 10.1007/978-3-319-67558-9_28.
- [26] Garbin, S. J., Shen, Y., Schuetz, I., Cavin, R., Hughes, G., & Talathi, S. S. (2019). OpenEDS: Open Eye Dataset. arXiv preprint arXiv:1905.03702. doi: 10.48550/arXiv.1905.03702
- [27] Land, M. "Saccade," Encyclopedia Britannica, Aug. 20, 2012. Available: <https://www.britannica.com/science/saccade>.
- [28] Ronneberger, O., Fischer, P., & Brox, T. "U-Net: Convolutional Networks for Biomedical Image Segmentation," arXiv preprint arXiv:1505.04597, May 2015. doi: 10.48550/arXiv.1505.04597.
- [29] Shahoveisi, F., Taheri Gorji, H., Shahabi, S., et al. "Application of image processing and transfer learning for the detection of rust disease," Scientific Reports, vol. 13, no. 5133, 2023. doi: 10.1038/s41598-023-31942-9.
- [30] GeeksforGeeks, "DenseNet explained," GeeksforGeeks, Available: <https://www.geeksforgeeks.org/densenet-explained/>.
- [31] Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. "Densely Connected Convolutional Networks," arXiv, Jan. 2018. doi: 10.48550/arXiv.1608.06993.
- [32] Palmero, C., Sharma, A., Behrendt, K., Krishnakumar, K., Komogortsev, O. V., & Talathi, S. S. "OpenEDS2020 Challenge on Gaze Tracking for VR: Dataset and Results," Sensors (Basel), vol. 21, no. 14, p. 4769, Jul. 2021. doi: 10.3390/s21144769.
- [33] Tobii AB. *Tobii Pro Spectrum User Manual v2.9*. SDS Associates; 2023. Mean end-to-end latency <2 ms at 1200 Hz.
- [34] SR Research Ltd. *EyeLink 1000 Plus User Manual (Version 1.52)*. NatMEG; 2013.
- [35] Kassner F, Patera W, Bulling A. Pupil: An Open Source Platform for Pervasive Eye Tracking and Gaze-based Interaction. In: *Proceedings of UbiComp '14*; 2014. p. 1151-1160. <https://doi.org/10.1145/2638728.2641695>
- [36] Pupil Labs. *IR Eye Tracking Camera Accessory*. Pupil Labs; n.d. Available at: <https://pupil-labs.com/products/core/accessories> (accessed April 2025).
- [37] Saunders DR, Woods RL. Direct measurement of the system latency of gaze-contingent displays. *Behav Res Methods*. 2013;45(1):130-147. <https://doi.org/10.3758/s13428-012-0244-0>
- [38] Loschky, L. C. and Wolverton, G. S. 2007. How late can you update gaze-contingent multi-resolutional displays without detection? *ACM Trans. Multimedia Comput. Comm. Appl.* 3, 4, Article 25 (December 2007), 10 pages. <http://doi.acm.org/10.1145/1314303.1314310>
- [39] Duchowski AT, Cournia N, Murphy H. Gaze-contingent displays: A review. *Cyberpsychol Behav*. 2004;7(6):621-634. <https://doi.org/10.1089/cpb.2004.7.621>

APPENDIX

A. User Interface

To make our model easier to interact with for various applications, we developed a Graphical User Interface (GUI) with key functionalities. Refer to the figures in this appendix for visuals of GUI operation.

1) Segment eye images in postprocessing

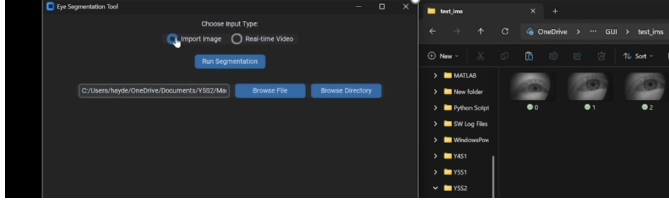
There are two methods built in for segmenting eye images in postprocessing. First, one can upload a single eye image by clicking browse file, or second one may segment a directory of eye images at a time by clicking brows directory as seen in figure 19a.

2) Segment real time video

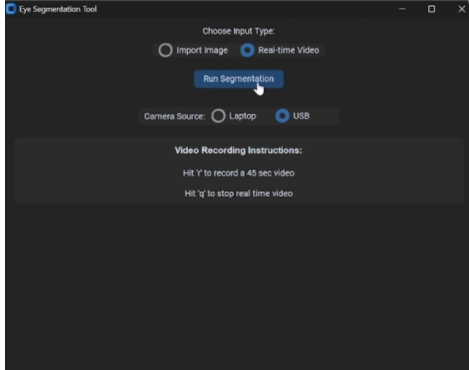
As seen in figure 19b, if one elects to process camera videos in real time, they have the option between a built in desktop camera or to attach an external USB webcam.

3) Save raw and segmented video frames

Once the user clicks 'Run Segmentation' to save frames they simply must press 'r' and the program saves a timestamped folder to the directory of their choice. This folder contains 45 seconds of raw frames along with their segmented counterparts. These two frames also appear in a real time window as seen in 19c.



(a) Post processing segmentation



(b) Real time segmentation UI



(c) Webcam view of real time segmentation

Fig. 19: SERTnet GUI